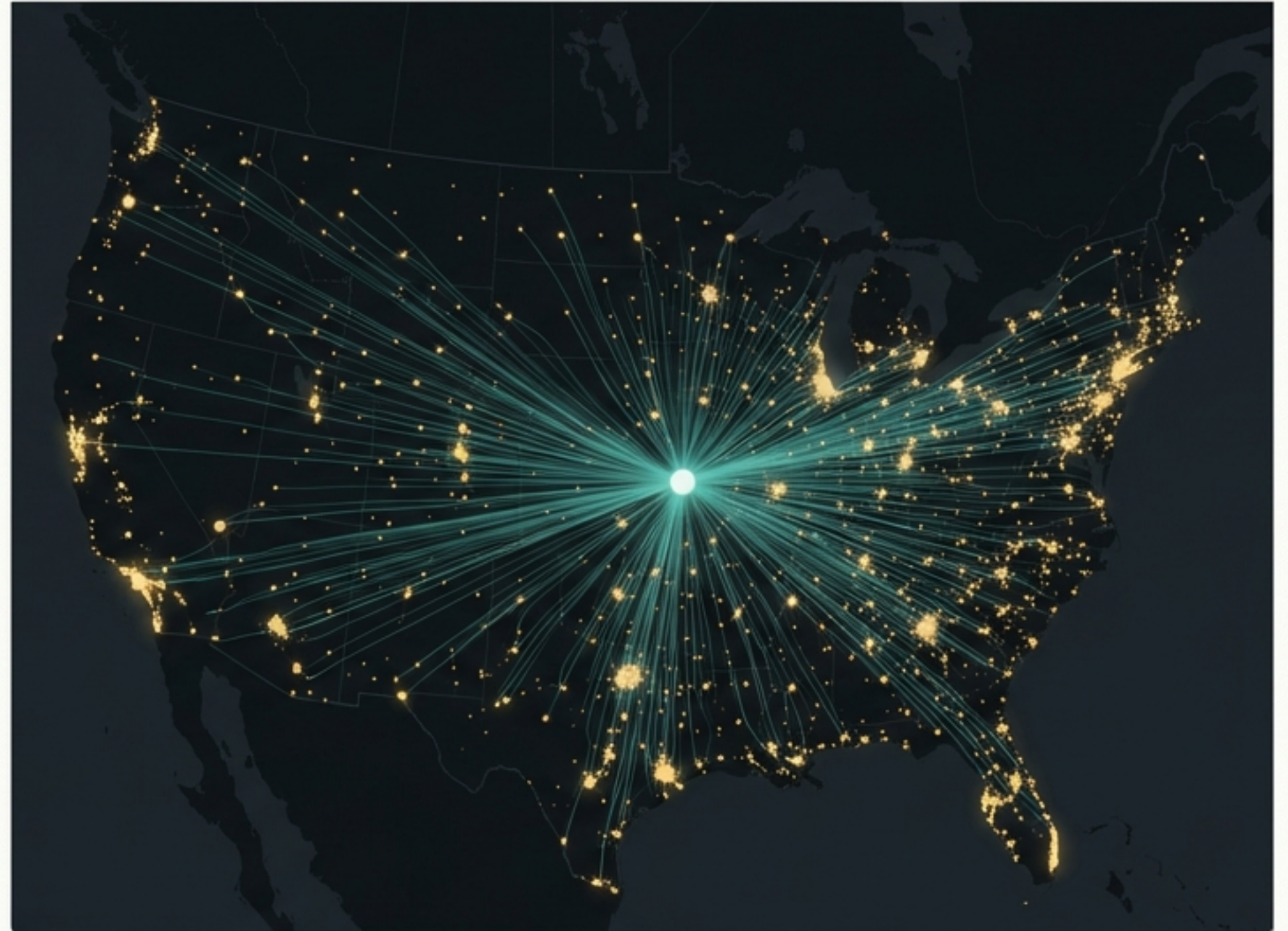


Detecting Vehicle Crashes in Real-Time Across a Fleet of 1 Million Trucks

An architectural overview of the IoT stream processing system that powers mission-critical safety alerts.



1,000,000

Vehicles

< 5

Seconds p99 Latency

4.5

GB/s Peak Throughput

100+

Provider Integrations

From Data Chaos to Actionable Intelligence

The Business Imperative

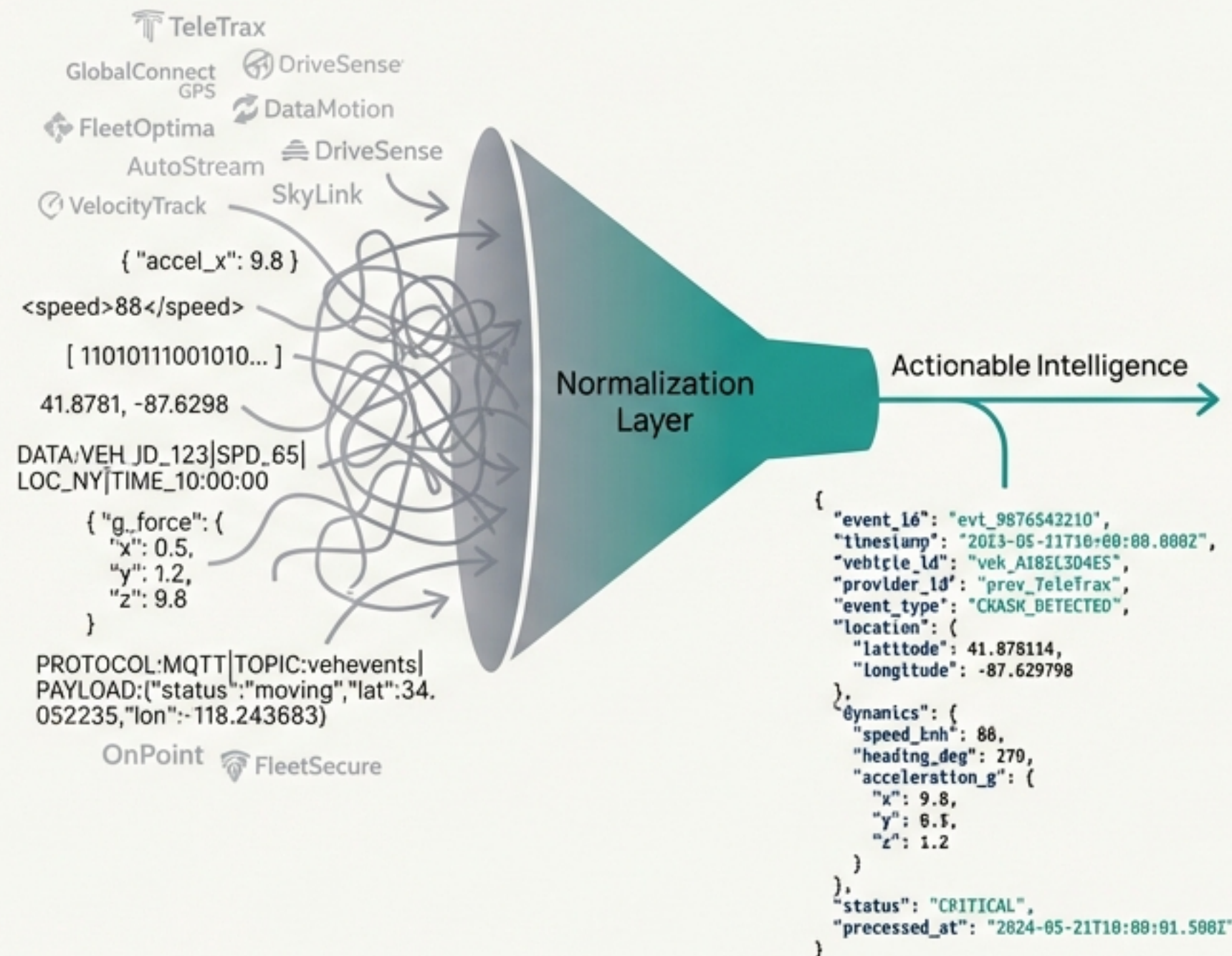
Problem: Manual crash detection causes critical delays in emergency response and inflates claims processing costs.

Goal 1: Immediate Detection.

Detect crashes in **< 5 seconds** to save lives and assets.

Goal 2: Faster Settlement.

Automatically pre-populate claims data to reduce processing time by **60%**.

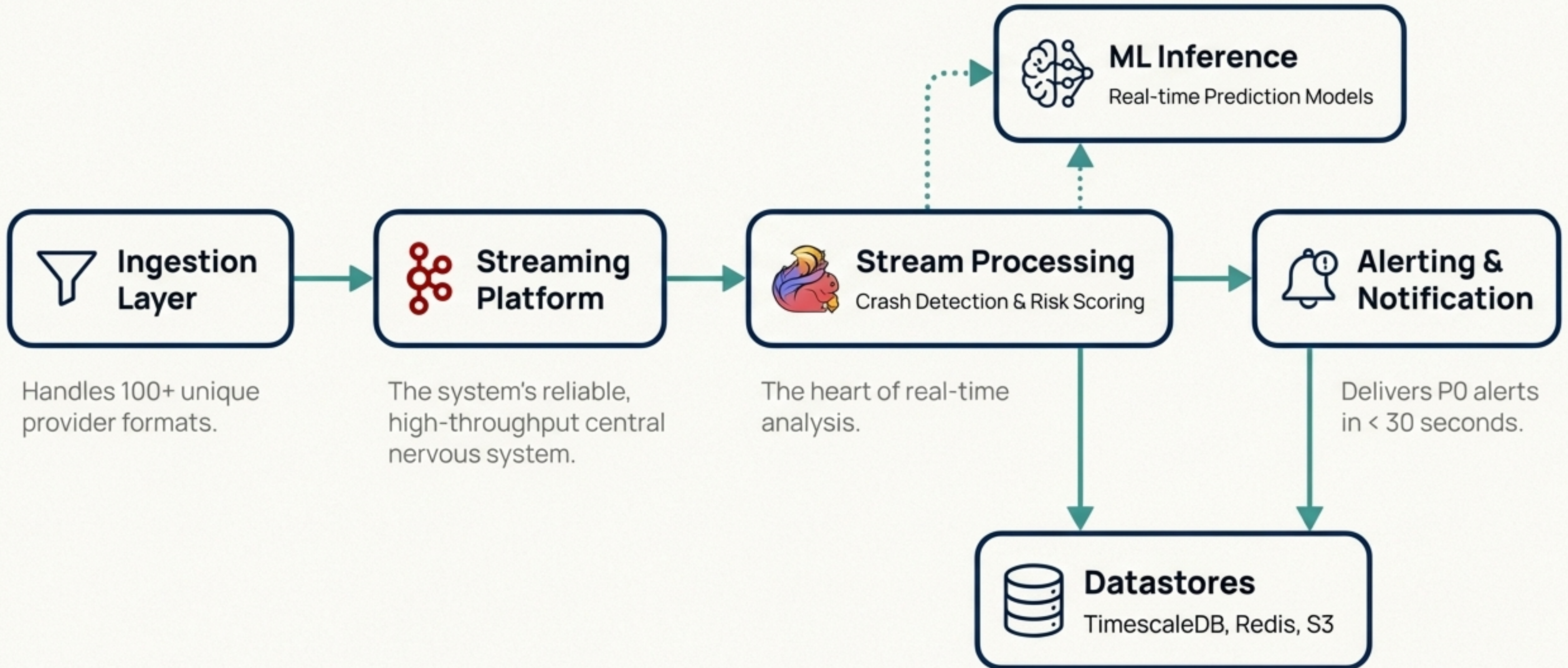


The Technical Challenge

Problem: Ingesting and normalizing data from over 100 different telematics providers, each with unique APIs, protocols, and data formats. Key complexities:

- Push vs. Pull integration models.
- Massive data volume (**~260 TB/day**).
- Unreliable real-world networks (cellular gaps, latency).

An Architecture Built for Speed, Scale, and Resilience



A Purpose-Built Stack for Real-Time IoT

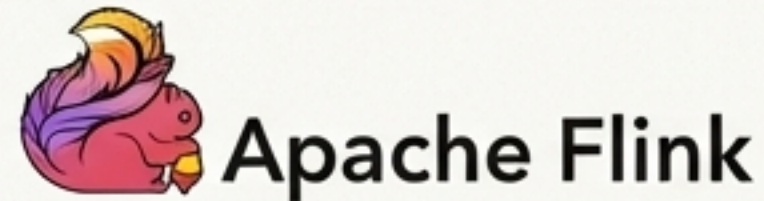
Message Queue



Apache Kafka

Chosen for its extreme throughput, reliability, and partitioning capabilities (300 partitions keyed by `vehicle\_id`).

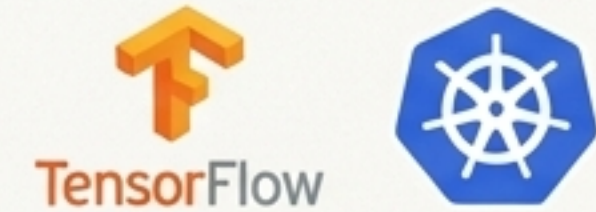
Stream Processing



Apache Flink

Chosen for its advanced state management, event-time processing, and windowing capabilities, essential for accurate time-series analysis.

ML Serving



TensorFlow Serving on Kubernetes

For low-latency (< 50ms p99) inference, serving models that detect complex crash patterns beyond simple rules.

Time-Series Storage



TimescaleDB

For efficient storage and querying of the massive raw telemetry stream (7-day hot retention).

Caching



Redis Cluster

Used for maintaining real-time vehicle state, deduplication sets, and rate limiting.

Observability



Prometheus + Grafana + Loki

A comprehensive stack for deep monitoring of every component in the system.

Inside the Detection Engine: Processing 9 Million Events Per Second

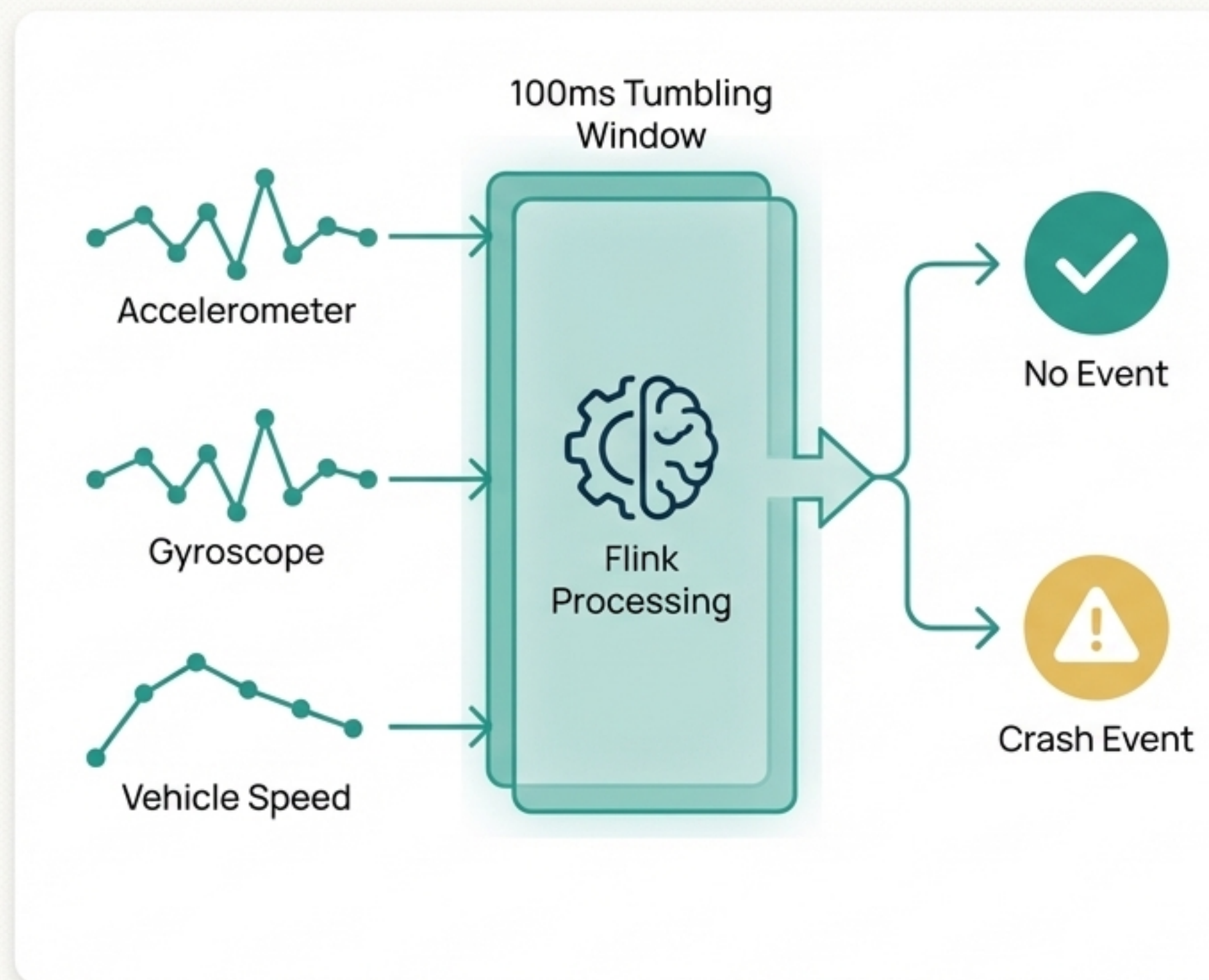
Event Time Processing

We use Flink's Event Time (based on sensor timestamps) not Processing Time. This is critical because for crash analysis, we must know when events actually occurred, not just when they arrived.

Multi-Signal Fusion

The algorithm isn't a simple rule. It fuses multiple data points within a 100ms tumbling window to build a high-confidence picture of a crash event.

Partitioning all data by `vehicle\_id` ensures that all data for a single truck is processed on the same node, maintaining state and context.



The Real World is Messy: Our Strategy for Handling Late Data

In a safety-critical system, data delayed by network issues (from cellular gaps to provider outages) cannot be dropped. Every event is potentially part of a crash signature.

Scenarios That Cause Late Data	
Scenario	Typical Lateness
Network Jitter	< 1 second
Cell Coverage Gaps	1-30 seconds
Provider API Downtime	1-60 minutes
Extended Connectivity Loss	Hours to Days

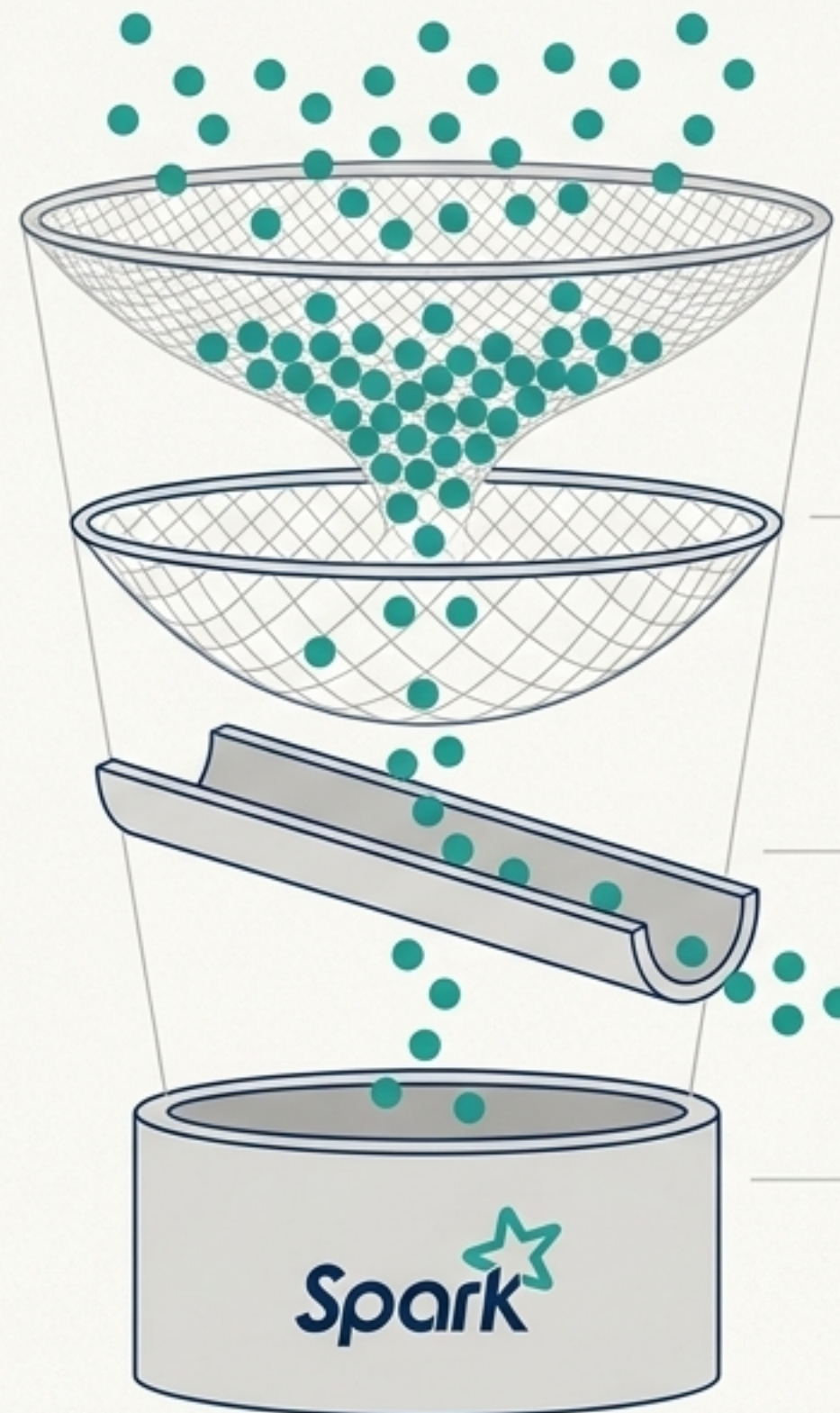
NEVER DROP DATA. Late events are valuable and must always be processed, even if through different paths.

A 4-Layer Defense Against Data Lateness

Layer 1: Watermark Out-of-Orderness

For: Normal network jitter (< 5 seconds).

Flink's watermarks are configured to tolerate events arriving up to 5 seconds out of order, automatically placing them in the correct time window.



Layer 2: Allowed Lateness

For: Temporary gaps in cell coverage (< 30 seconds).
Windows are allowed to fire a second time if a late event arrives within 30 seconds of the watermark passing.
Downstream systems handle the updated result.

Layer 3: Side Output

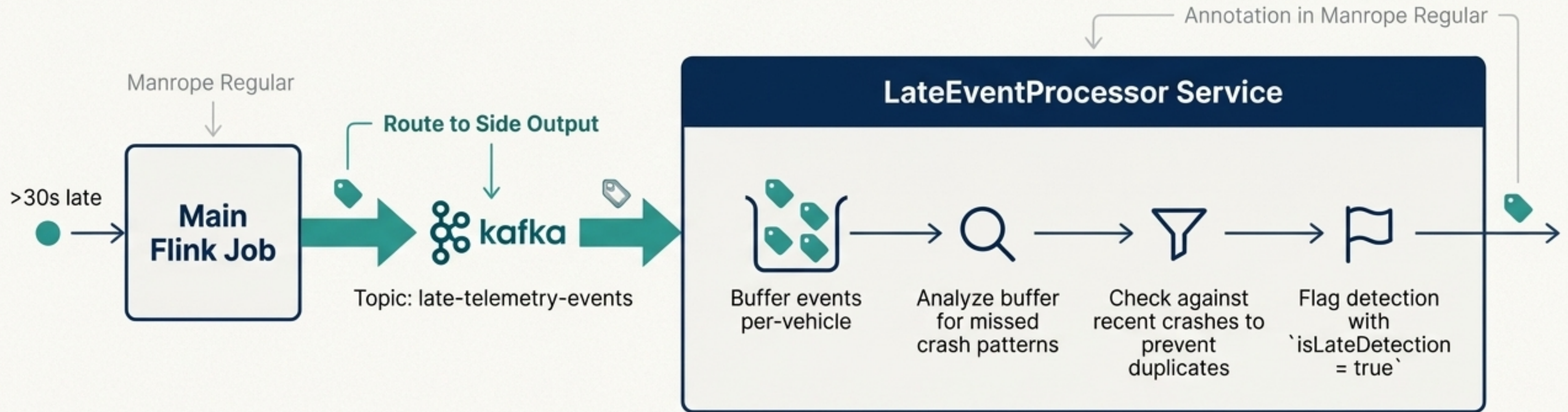
For: Extended provider delays (< 60 minutes).
Events arriving after the allowed lateness are not dropped. They are routed to a separate "Side Output" stream for dedicated processing.

Layer 4: Batch Reprocessing

For: Catastrophic, multi-hour outages.
Data is safely stored in S3 and reprocessed later using a Spark batch job to find any missed incidents.

Deep Dive on Layer 3: The Late Event Side Output

To ensure that even significantly delayed data (30s to 60min) can still result in a valid crash detection. This is too valuable to ignore.



This ensures we can still file a claim and analyze an incident, even if the real-time alert window was missed.

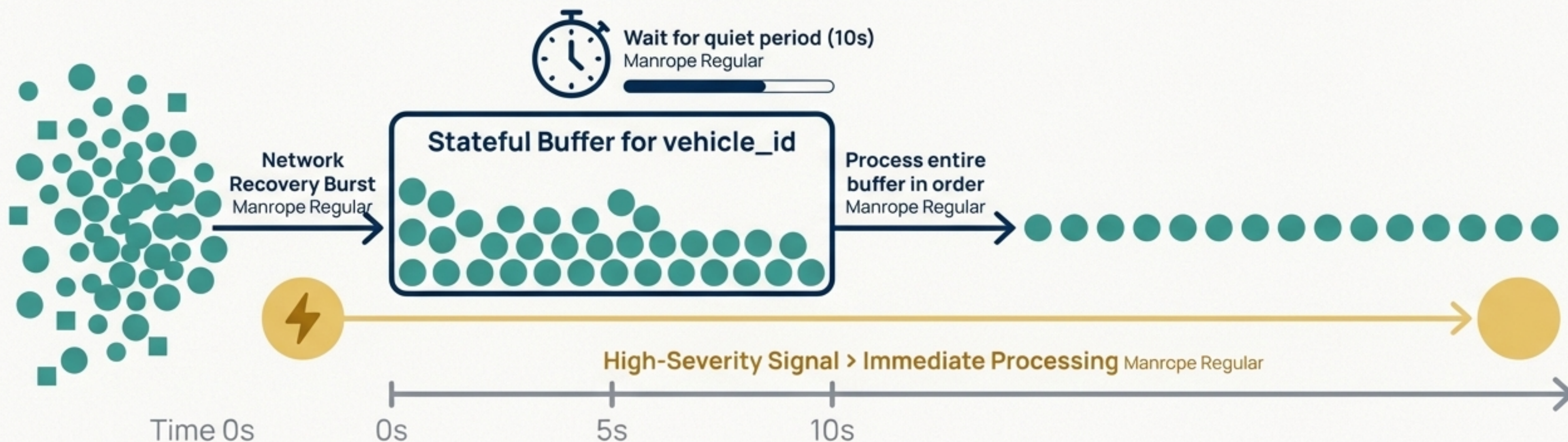
Surviving the Storm: Handling Network Recovery Bursts

The Challenge

When a truck with hours of stored data regains connectivity, **it can upload thousands** of events in seconds. This “thundering herd” can overwhelm a naive processing system.

Our Solution

An intelligent, timer-based buffering strategy within the Flink job.



Delivering the Signal: From Detection to Action in Seconds

The Goal: A confirmed crash detection must translate into a stakeholder notification within 30 seconds.

Alert Priority Classification			
Priority	Criteria	Response Time	Channels
P0-Critical	Confirmed crash, severity ≥ 3	< 15 seconds	SMS + Voice + Push + Dashboard
P1-High	Confirmed crash, severity 1-2	< 30 seconds	SMS + Push + Dashboard
P2-Medium	Probable crash (confidence 65-85%)	< 2 minutes	SMS + Dashboard
P3-Low	Risk alert, near-miss event	< 5 minutes	Push + Dashboard



We Don't Just Aim for Speed, We Guarantee It: Our Tier 1 SLAs

SLA	Target	Measurement
Crash Detection Latency	p99 < 5 seconds	Time from sensor event to `crash-events` topic.
P0 Notification Delivery	p95 < 30 seconds	Time from detection to SMS delivery confirmation.
System Availability	99.95% uptime	Availability of the core detection & notification path.
ML Inference Availability	99.9% uptime	ML service successfully responding to requests.

These SLAs are monitored in real-time and trigger SEV1 incidents if breached, ensuring the highest level of reliability for our life-saving service.

If It Moves, We Measure It: Comprehensive System Observability

We cannot guarantee reliability without total visibility into every component.



1. Ingestion Health

ingestion_latency_seconds
provider_availability
dlq_events_total



2. Stream Processing

flink_processing_latency_ms
kafka_consumer_lag
watermark_lag_seconds



3. ML Inference

ml_inference_latency_ms
ml_false_positives_total
gpu_utilization_percent



4. Alert & Notification

alert_detection_to_notify_seconds
notification_failed_total

All metrics are collected in Prometheus, visualized in Grafana, with automated alerting tied directly to our incident response runbooks.

An Engine for Certainty in an Uncertain World

From over 100 chaotic provider streams, this system forges a single, reliable stream of truth. By embracing the complexity of real-world data and building a resilient, multi-layered architecture, we can turn a raw 4.5 GB/s of telemetry into a life-saving alert in under 5 seconds. It is a system built not just for scale and speed, but for absolute reliability when it matters most.

